

The background features a dark blue, stylized illustration of a drone swarm. Several drones, depicted as small cubes with four propellers, are scattered across the frame. They are interconnected by a network of thin, light blue lines, suggesting a communication or coordination mesh. The overall aesthetic is technical and futuristic.

Resilient Multi-Agent Drone Coordinator

in Rust with Docker-Based Deployment

Project Overview

Our core objective is to develop a **fault-tolerant coordination system** for a drone swarm. This system leverages **ROS2** and **Gazebo** for simulation and control, with its underlying logic implemented using a Rust actor-based runtime.

Our key technical approach relies on **asynchronous message** passing for robust inter-drone communication and a **leader election** mechanism to ensure fault tolerance and **system stability**.

The deployment architecture utilizes **Docker containers** for each component, ensuring isolated, scalable, and reproducible environments across the drone swarm.

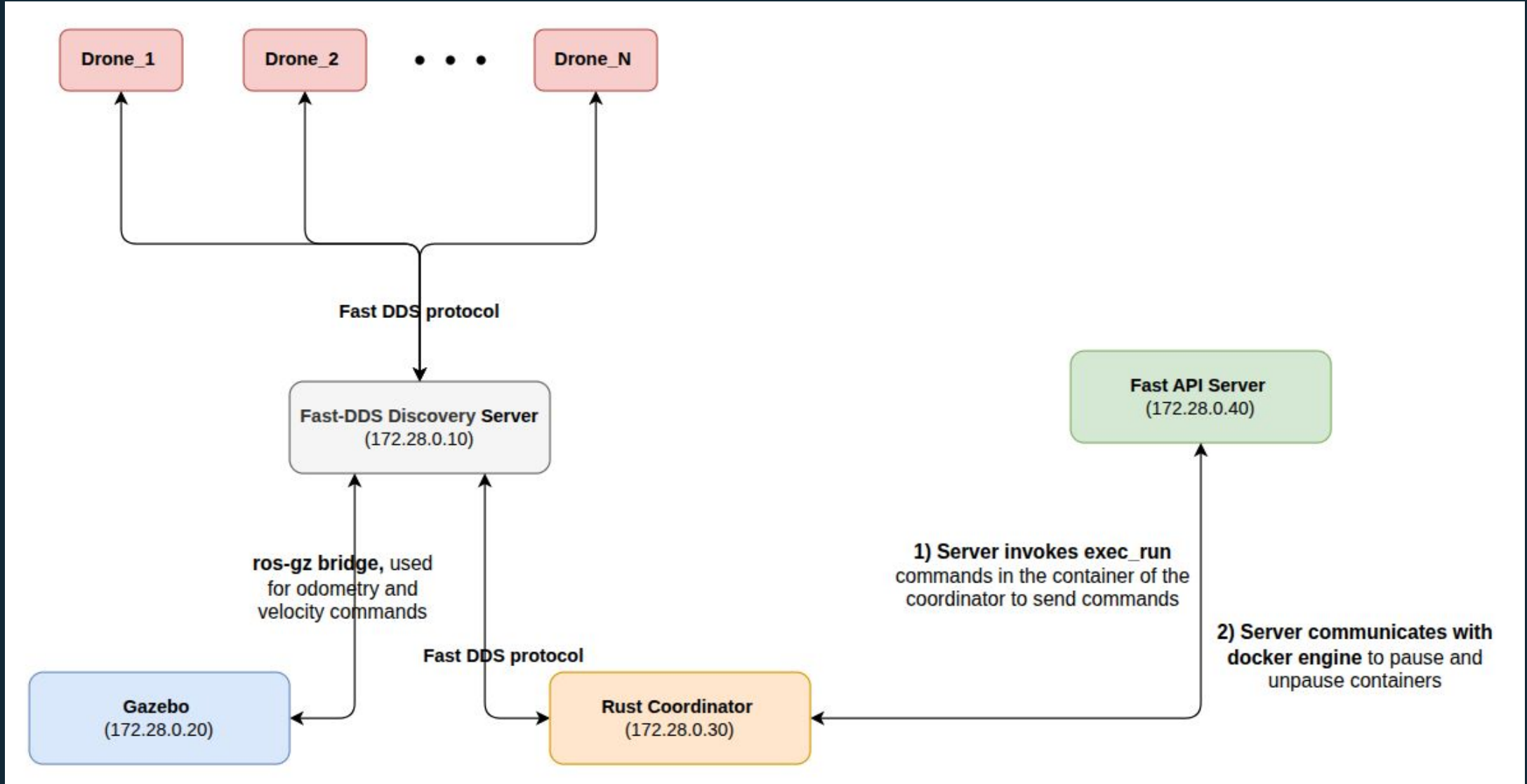
For observability and control, the system includes a **FastAPI backend**, providing a REST interface for monitoring drone status, managing tasks, and receiving system alerts.

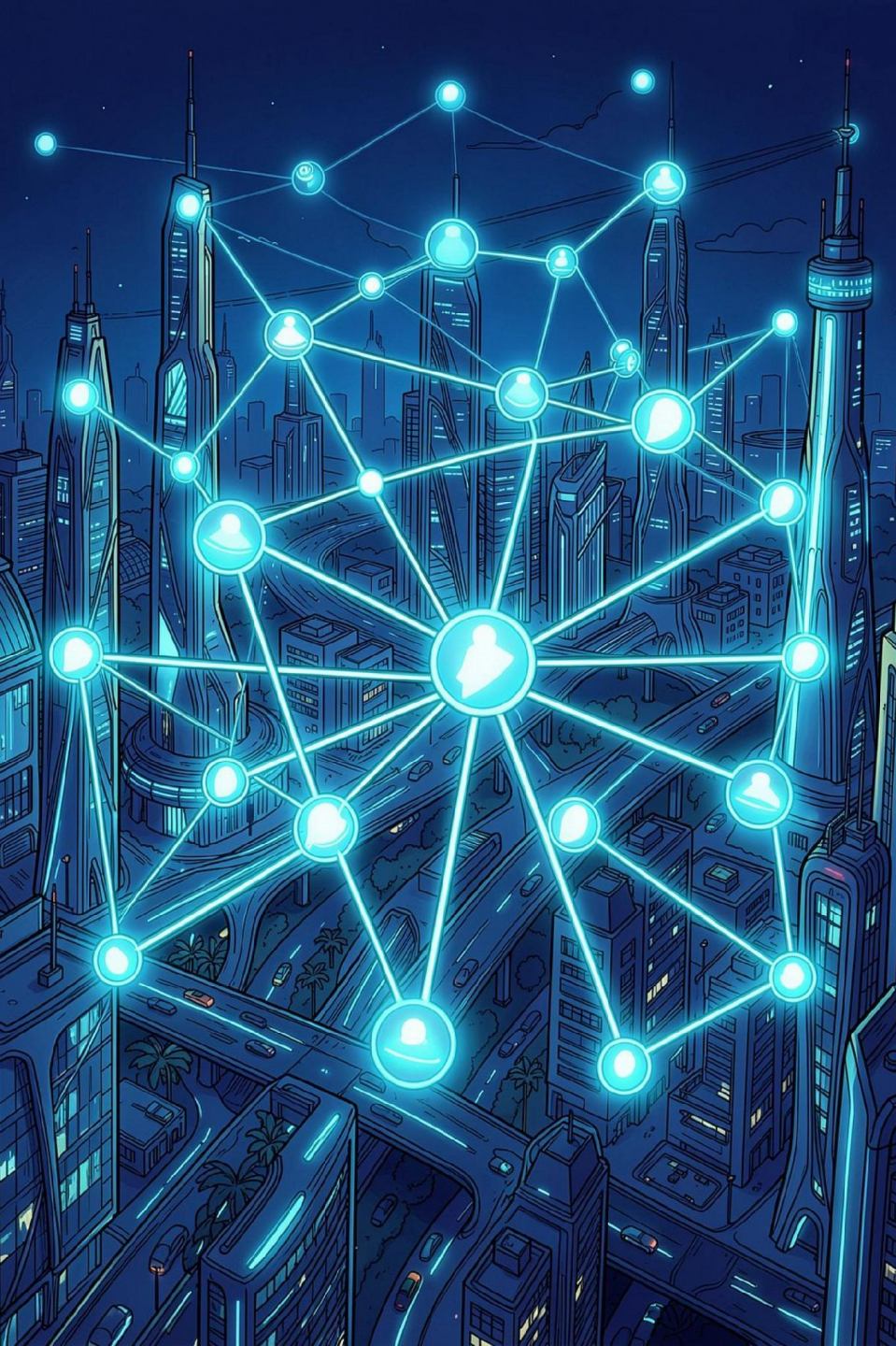


Expected Deliverables

- Rust based coordinator
- ROS2 node wrappers for each drone agent
- Gazebo simulation scenario
- Docker Compose deployment
- FastAPI observability backend
- Formal model of the FSM
- Public GitHub repository with documentation

Architecture:





DDS and Communication Architecture

Swarm of drones Coordinator — Data Distribution Service for real-time distributed systems

OMG STANDARD

REAL-TIME SYSTEMS

IIOT

Formal Definition

What is DDS?

DDS (Data Distribution Service for Real-Time Systems) is an OMG (Object Management Group) standard that defines a **data-centric publish/subscribe** communication middleware for real-time distributed systems.

There are nowadays several variants of this protocol, like the wired one (RTPS).

- ❏ In this project we use **FastDDS based on eProsima**, an open-source C++ implementation of the DDS standard (Apache 2.0 license). It implements the RTPS wire protocol and is the default middleware for ROS 2. It provides configurable real-time publish-subscribe communication with shared memory transport, multiple discovery mechanisms, and high-performance data exchange.

History & Adoption

From Military Protocol to Global Standard



1

1990s

Military and aerospace systems with 15+ incompatible proprietary middleware solutions.

2

2001–2003

OMG standardization. First DDS 1.0 specification published in 2003.

3

Implementations

RTI Connext DDS (F-35), OpenSplice (defense), FastDDS (ROS2 default).

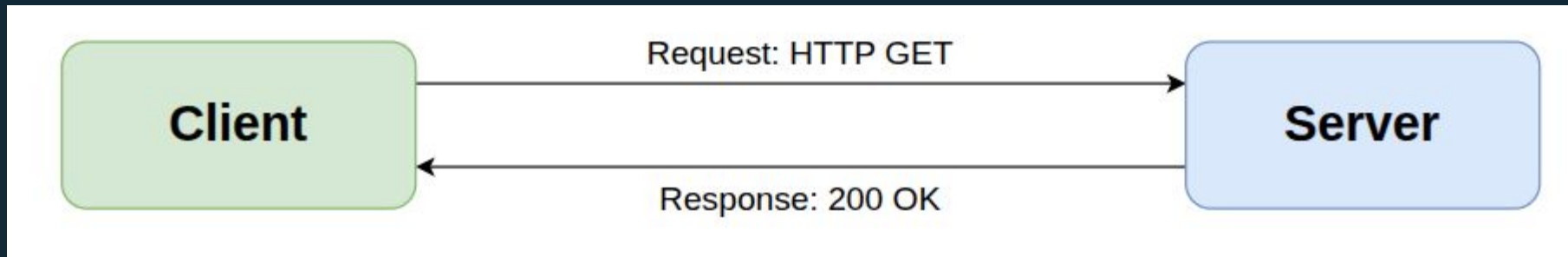
4

Today

Aerospace, automotive (AUTOSAR Adaptive), robotics (ROS2), smart grid, healthcare.

Client-Server Architecture: Request-Response Pattern

The client asks and the server responds. The pattern is synchronous request-response: every communication is actively initiated by whoever needs the data.



The server is passive with respect to real data — it waits for someone to request it.

While variants like WebSocket/SSE (persistent channels) and MQTT-style brokers (centralized pub/sub) exist, the core characteristic of client-server remains that all communication flows through a central point.

❏ The core limitation is that **all communication flows through a central point.**

The 4 Main Limits of Client-Server Architecture

Problem	DDS Solution
Single Point of Failure If the Server/Broker crashes, the entire system stops instantly.	Native Peer-to-Peer :There is no central node. If one node fails, the others continue communicating without any interruption.
Bottleneck (Latency and Bandwidth) All traffic must pass through the server. As data increases, the server struggles.	Direct Communication :Data goes straight from the producer to the consumer. No intermediate steps, minimal latency, and high scalability.
Rigid Coupling The Client must know the Server's IP/port to send or request data.	Data-Centricity :Nodes don't ask "Who are you?", but rather "Who is interested in Topic X?". They publish and receive data without knowing each other's address.
"All or Nothing" Service Management It is difficult to manage different requirements for different data on the same channel.	Granular QoS (Quality of Service) :You can define specific rules for each data stream (e.g., reliability, frequency, persistence).

DDS Architecture

Domain & DomainParticipant

Domain

A logical isolation channel identified by a **Domain ID**.

Participants with the same ID form a communication "bubble" invisible to other domains.

DomainParticipant

It is the **root entity** of every DDS application. It represents an app's presence in the domain and acts as a factory for **Publishers, Subscribers, and Topics**.

Domain 0

Smart City: Sensors ↔ Backend ↔
Traffic Lights

Domain 1

Maintenance: Technicians ↔
Diagnostics

Domain 2

Emergency: 112 ↔ Alerts

DDS Architecture

Topic, Publisher & Subscriber



Topic

Three attributes: **Name** (unique within the domain), **Type** (data structure, type-safe), **QoS** (quality policies).



Publisher

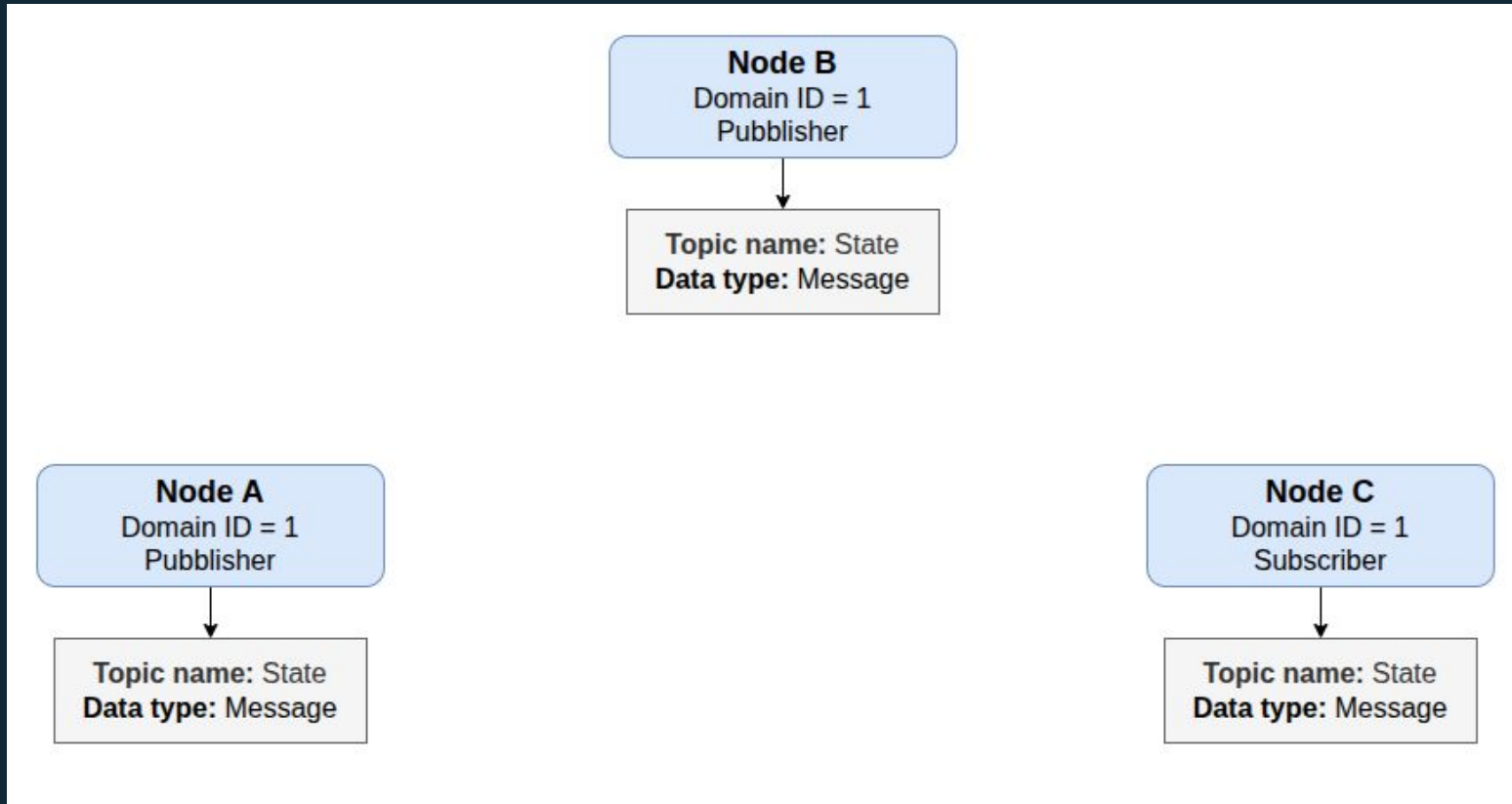
Grouping entity for DataWriters. Manages shared QoS. Can contain multiple DataWriters on different topics.



Subscriber

Grouping entity for DataReaders. Analogously to the Publisher, it defines default QoS for child readers.

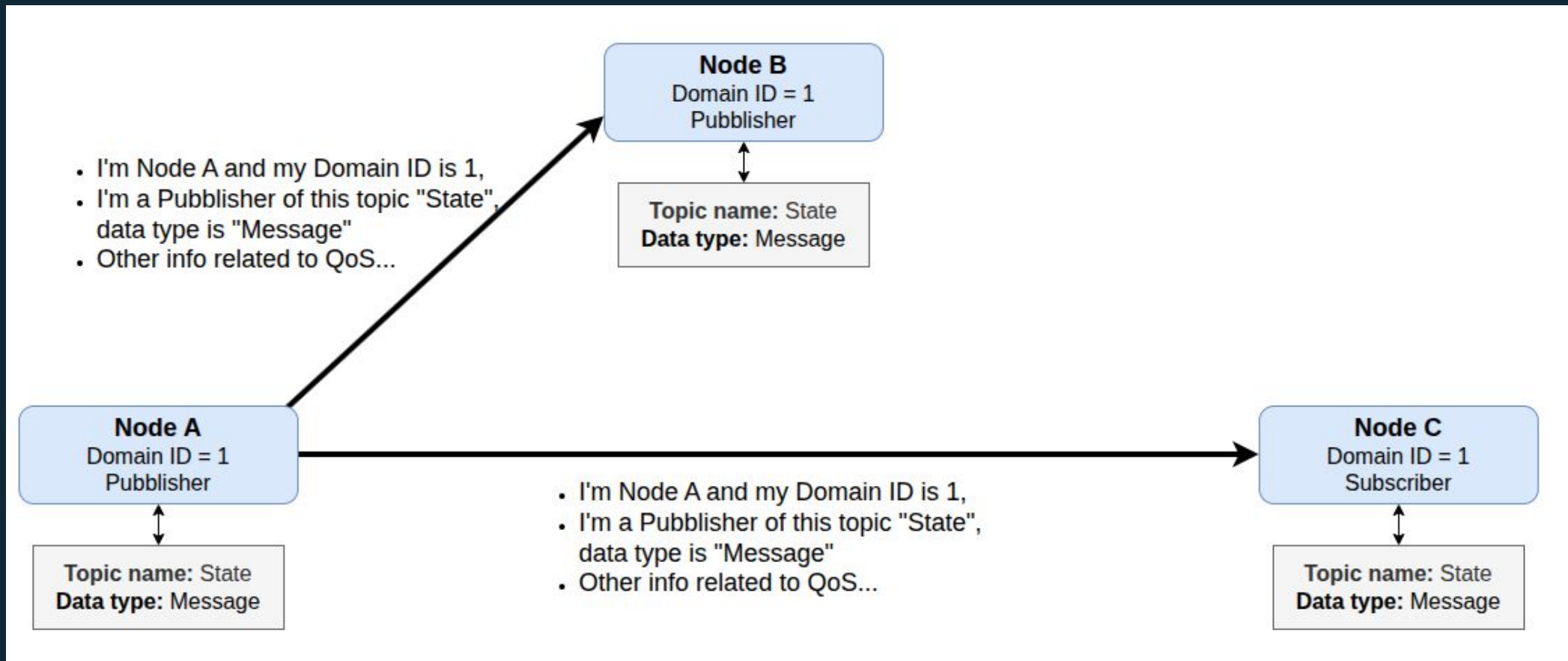
Example - 3 nodes in the Network:



DDS Architecture

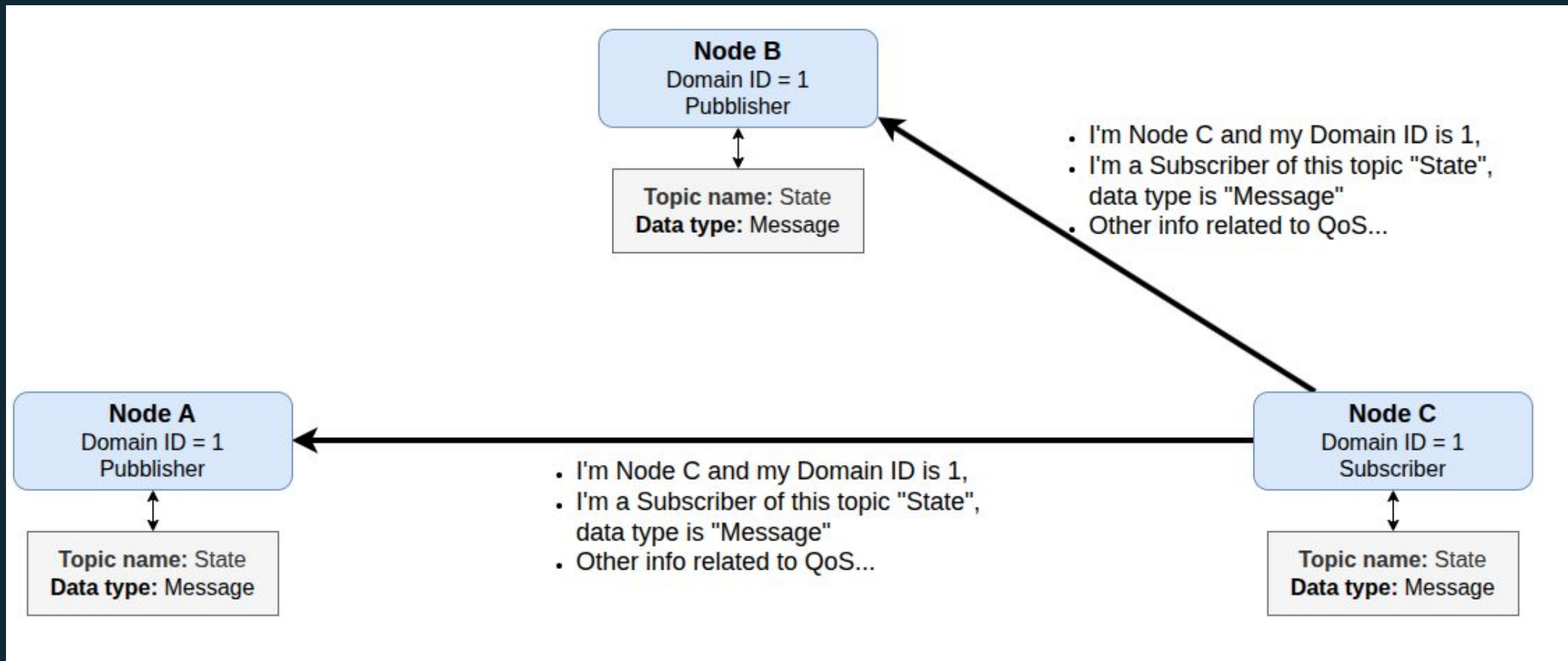
Example - 3 nodes in the Network:

Discovery phase starts, each node needs to be aware of all the other nodes that want to join in the DDS network, nodes start to send Multicast UDP packets to declare their presence to other nodes



Example - 3 nodes in the Network:

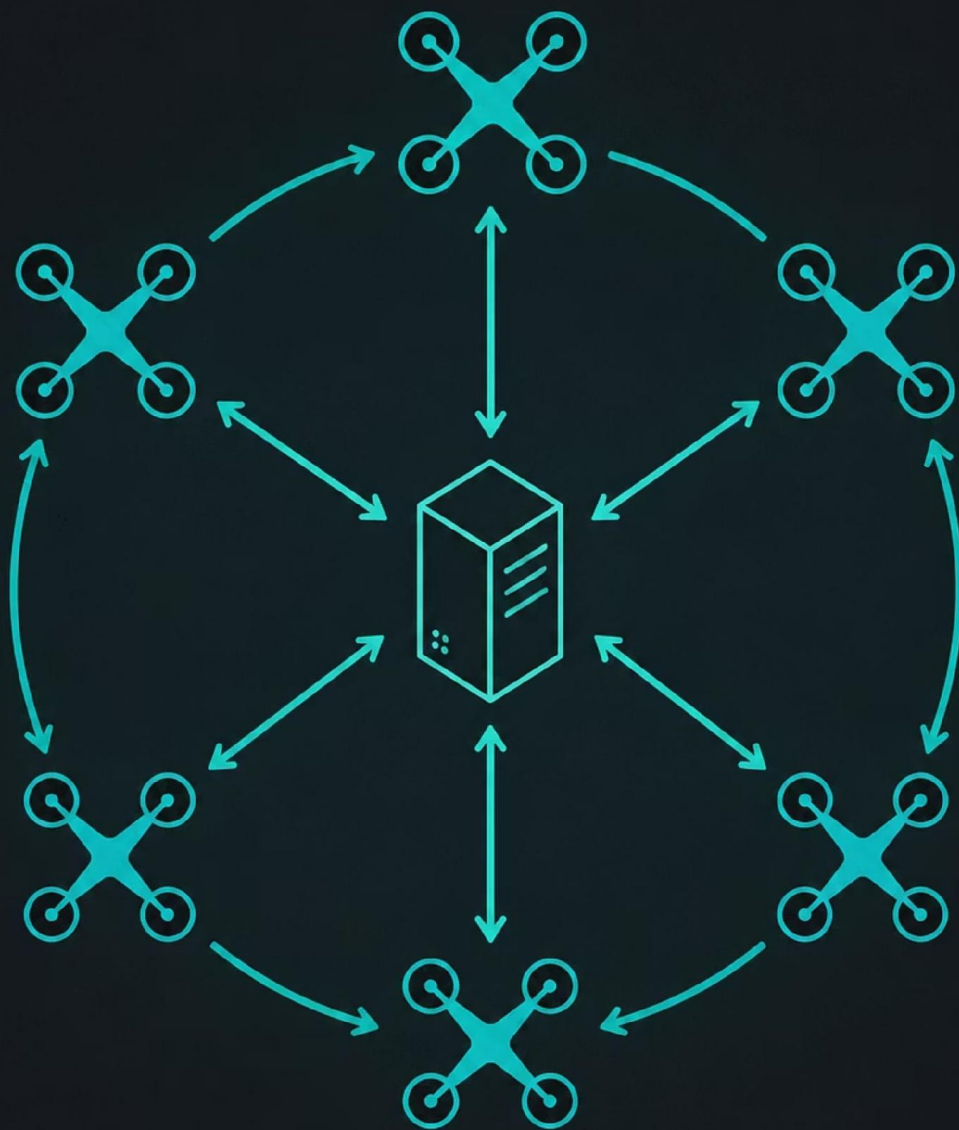
Discovery phase starts, each node needs to be aware of all the other nodes that want to join in the DDS network, nodes start to send Multicast UDP packets to declare their presence to other nodes



Example - 3 nodes in the Network:

During the Discovery Phase:

- Both nodes A and C belong to the same Domain ID, they want to write/read data on the same topic, the data type is the same → There is a Match!
- The Machine A detect this match, and they understand they can directly communicate, now Node A knows the Node C is a DataReader interested on the data posted by A → They can start to communicate directly.
- The same process is repeated for all the other nodes in the network.



Fast DDS

A modern C++ implementation of the OMG DDS standard

Fast DDS is a variant and implementation of the standard DDS (Data Distribution Service) specification, tailored for high-performance and real-time applications.

Developed by eProsima

Implements the Real-Time Publish-Subscribe (RTPS) protocol

Default middleware for ROS 2 Jazzy Jalisco

Provides low-latency, deterministic data exchange for robotic systems

Open-source (Apache 2.0 license)

Why Fast DDS in This Project?

Main Problem: Network Isolation

By default, ROS 2 relies on UDP Multicast for dynamic peer discovery.

Containerized microservice architectures using Docker bridge networks frequently block or restrict UDP multicast packets across isolated containers.

This creates network isolation issues.

The Solution: Fast DDS Discovery Server

Replaces multicast discovery with a centralized unicast architecture.

- Runs on port 11811
- Each drone agent and the Rust coordinator register directly with the Discovery Server upon container startup.
- Ensures deterministic, instant discovery of all ROS 2 nodes across Docker containers without packet loss or network isolation issues.



Key Benefit: Enables reliable peer-to-peer communication in containerized environments through a two-phase process: Server-Mediated Discovery followed by Direct Peer-to-Peer Data Communication.

Fast DDS Implementation

1) Definition of the service in the docker-compose.yml

```
discovery-server:
  image: ros:jazzy-ros-base
  container_name: discovery-server
  hostname: discovery-server
  command: >
    bash -c "source /opt/ros/jazzy/setup.bash && fastdds discovery -i 0 -l 0.0.0.0 -p 11811"
  networks:
    swarm_net:
      ipv4_address: 172.28.0.10
  ports:
    - "11811:11811/udp"
    - "11811:11811/tcp"
```

- A container based on the official ROS 2 Jazzy image is instantiated.
- The command `fastdds discovery -i 0 -l 0.0.0.0 -p 11811` is executed, starting the Discovery Server daemon on server ID 0, listening on port 11811 (both UDP and TCP).
- It is assigned the static IP address `172.28.0.10` within the `swarm_net` bridge network.

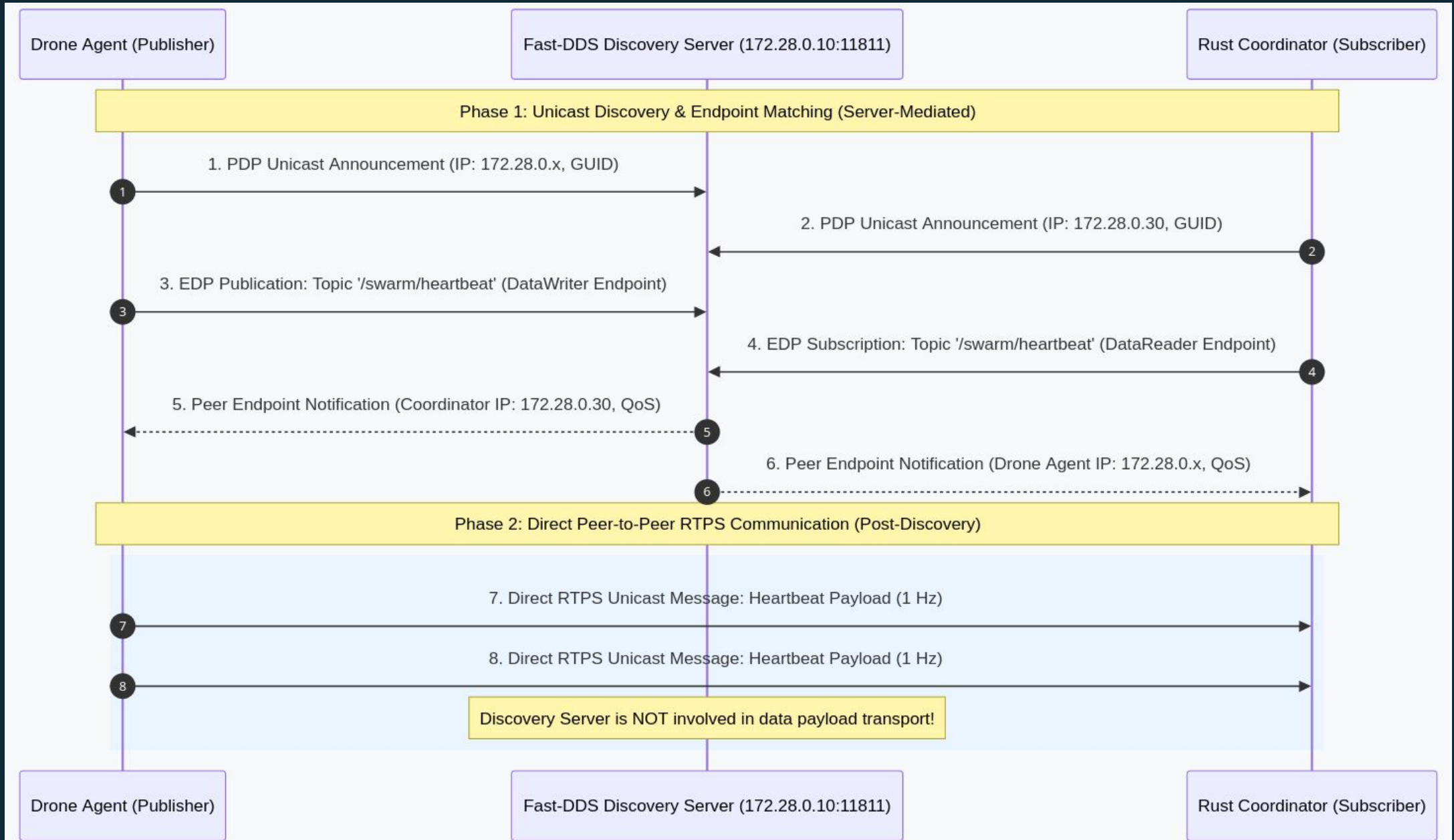
Fast DDS Implementation

2) Client Nodes Configuration

To ensure that the microservices connect to the Discovery Server instead of relying on standard Multicast discovery, a triad of environment variables is exported across all **entrypoint.sh** scripts:

- `RMW_IMPLEMENTATION=rmw_fastrtps_cpp`: Forces the use of the Fast-DDS middleware implementation.
- `ROS_DISCOVERY_SERVER=172.28.0.10:11811`: Provides the node with the Unicast IP address and port required for registration.
- `ROS_SUPER_CLIENT=true`: Grants the node full visibility (Super Client status) over the complete ROS 2 graph and all active topics across the swarm.

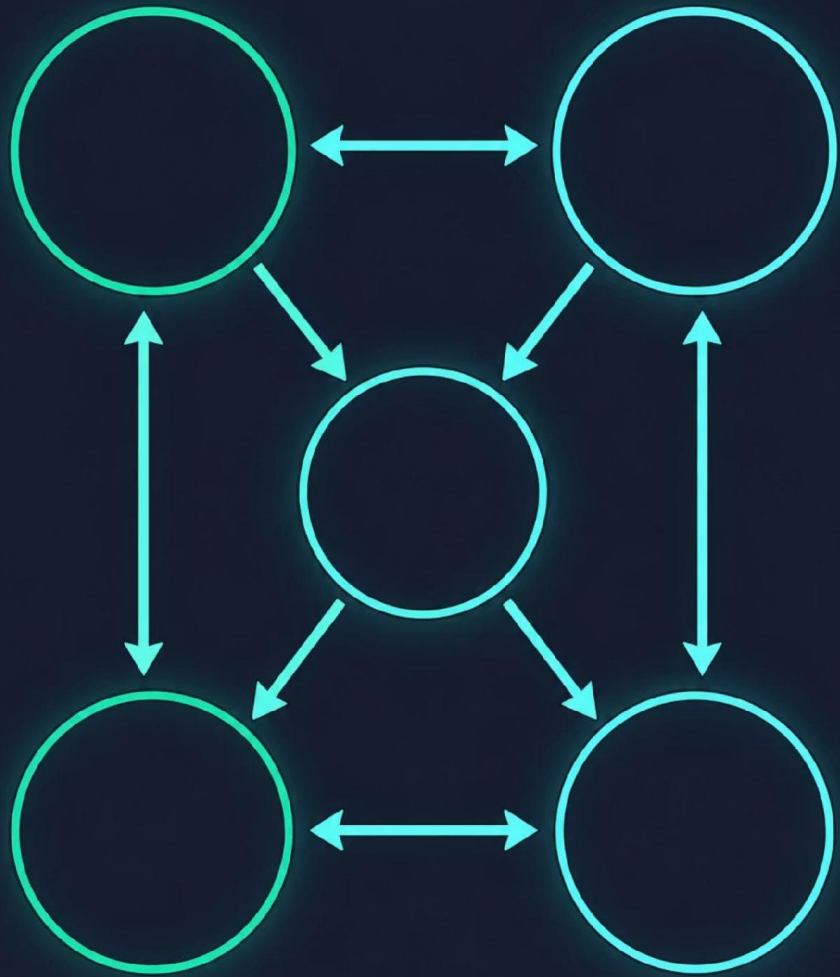
Fast DDS procedure:



How does Fast DDS works in simple terms:

Instead of shouting via Multicast hoping others will listen, a central node called the **Discovery Server** is created, serving as a *"Registry or Phonebook of Nodes"*:

1. **Container Registration:** Upon startup, every container sends a **Unicast** (direct, IP:Port `172.28.0.10:11811`) message to the Discovery Server stating: *"I am drone_1, this is my IP address, and I publish/subscribe to these topics"*.
2. **Endpoint Matching:** The Discovery Server matches nodes that need to communicate with each other (e.g., it relays `drone_1`'s IP address and endpoints to the Coordinator).
3. **Direct P2P Data Flow:** Once nodes have "discovered" each other, the **Discovery Server steps out of the data path**, and high-frequency data exchange (heartbeats, odometry, flight velocity commands) occurs directly **Peer-to-Peer** between containers with minimal latency.



Finite State Machine (FSM)

Drone states management — Managing all the possible states and transitions events

What is a Finite State Machine (FSM)?

A Finite State Machine (FSM) is a mathematical model of computation used to design systems that can be in only one state at a time from a finite set of predefined states, changing state (transition) in response to external events or inputs.

Key elements of an FSM

States

- The distinct conditions the system can be in
- Example: Off, On, Paused

Transitions

- The logical passage from one state to another
- Bound by precise rules

Events/Inputs

- The triggering factors that activate a transition
- External signals that cause state changes

Actions

- The behaviors executed by the system within or upon changing a state
- Operations performed during transitions

FSM Main Applications:



Video Games

Manage AI logic of characters

Example: Enemy transitions from Patrol → Chase → Attack states



Communication Protocols

Regulate the lifecycle of network connections

Example: TCP connection states (SYN_SENT, ESTABLISHED, FIN_WAIT)



Embedded and IoT Systems

Control physical machinery

Examples: Vending machines, elevators, HVAC systems, traffic lights



Compilers and Parsing

Lexical analysis to recognize syntactic structure

Used in source code parsing



User Interfaces (UI)

Manage complex navigation flows

Example: Idle, Loading, Success, Error states

Why FSMs Are So Useful

- Structural clarity: Replace long conditional chains with clean visual representation
- Predictability and safety: Prevent invalid states, every transition is rigidly coded
- Ease of maintenance: Scale by adding new states without altering entire architecture
- Bug resolution: Simplify debugging by identifying where logical flow breaks

Dual Finite State Machines for Drone Coordination

Every drone in the swarm is represented by a dual-state tuple (S1, S2) maintained by the Rust coordinator. This dual FSM approach separates concerns into two independent but complementary state machines.

FSM 1 - Health State (S1):

Tracks the operational status and connectivity of each drone:

- **Unregistered:** Drone newly spawned or initializing
- **Active:** Operational drone emitting periodic heartbeats
- **Suspected:** No heartbeat received for over 5 seconds
- **Failed:** No heartbeat received for over 10 seconds (drone considered crashed/offline)

FSM 2 - Swarm Role (S2):

Tracks the role and responsibility of each drone within the swarm:

- **None:** No role assigned (when S1 is Unregistered or Failed)
- **Candidate:** Eligible to become Leader during an election process
- **Follower:** Gregarious drone following the Leader's trajectory and formation directives
- **Leader:** Active leader drone governing the swarm

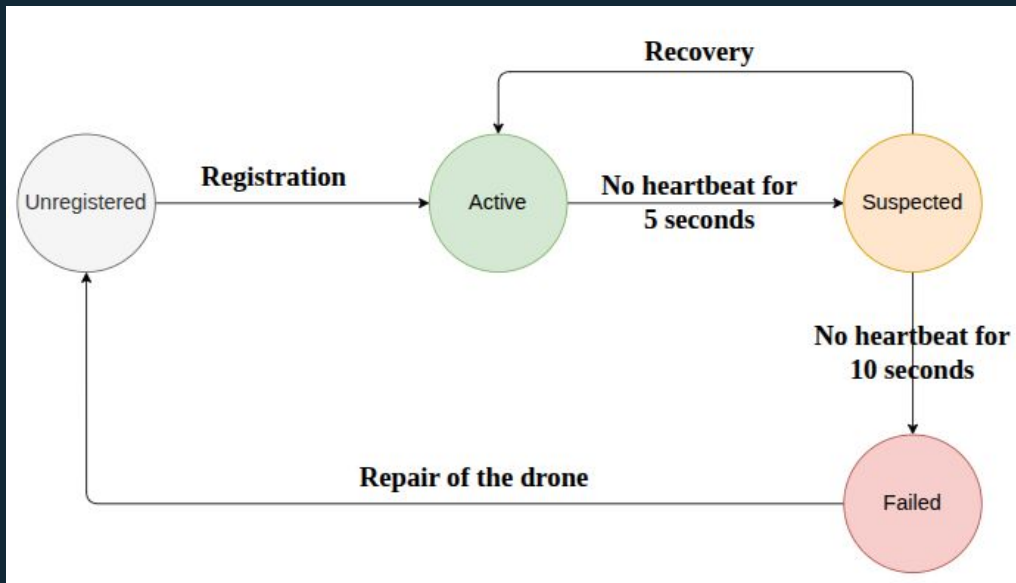
Why Dual FSM?

- **Separation of Concerns:** Health monitoring is independent from role assignment
- **Clarity:** Each FSM has a single responsibility, making the system easier to understand and maintain
- **Flexibility:** A drone can change roles without affecting its health state and vice versa
- **Robustness:** Failures in one dimension don't necessarily cascade to the other

FSM 1 & FSM 2 State Diagrams

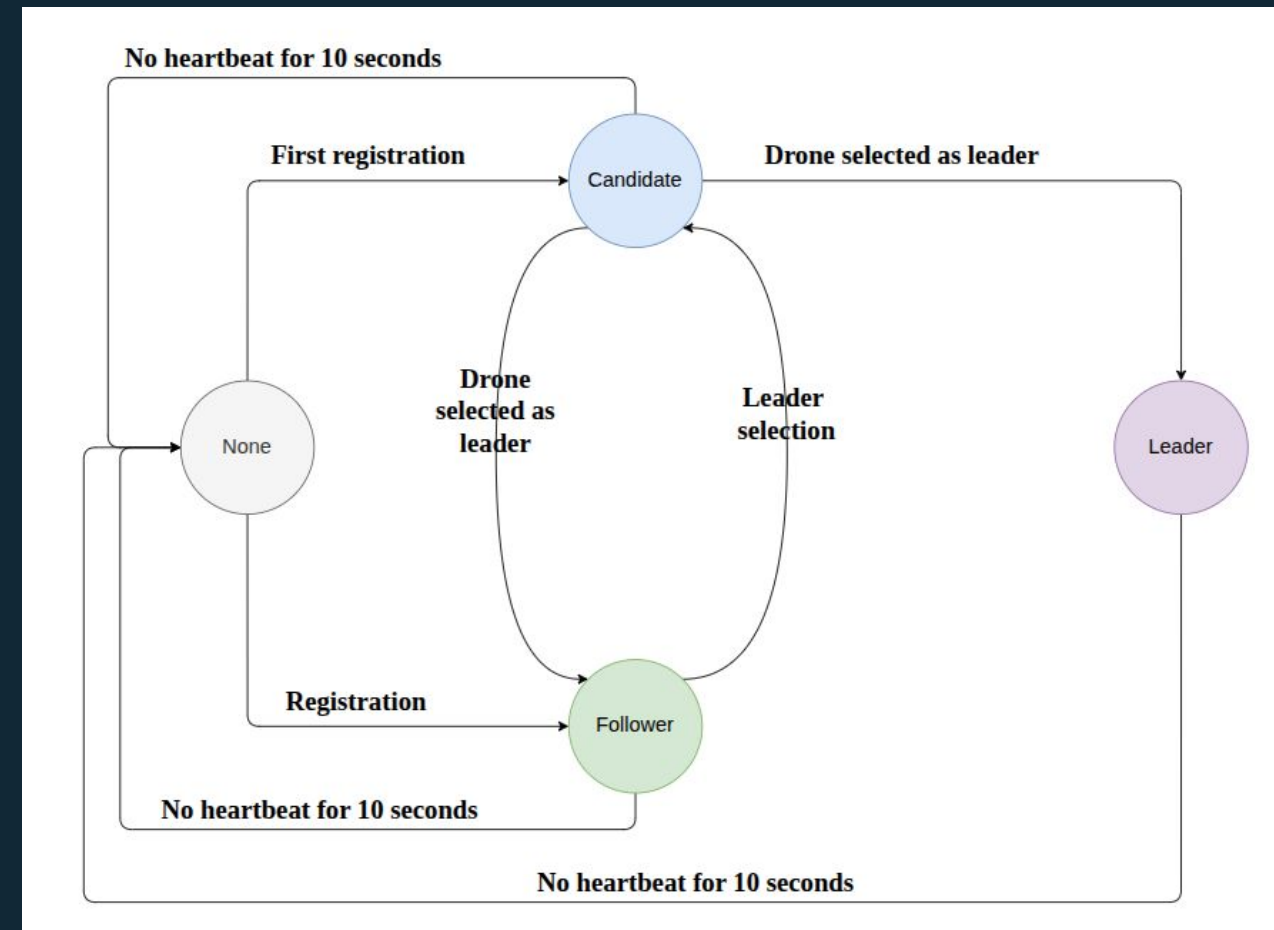
FSM 1: Health State

Drone operational status and connectivity



FSM 2: Swarm Role

Drone role within the swarm



Complete Dual-FSM Composition

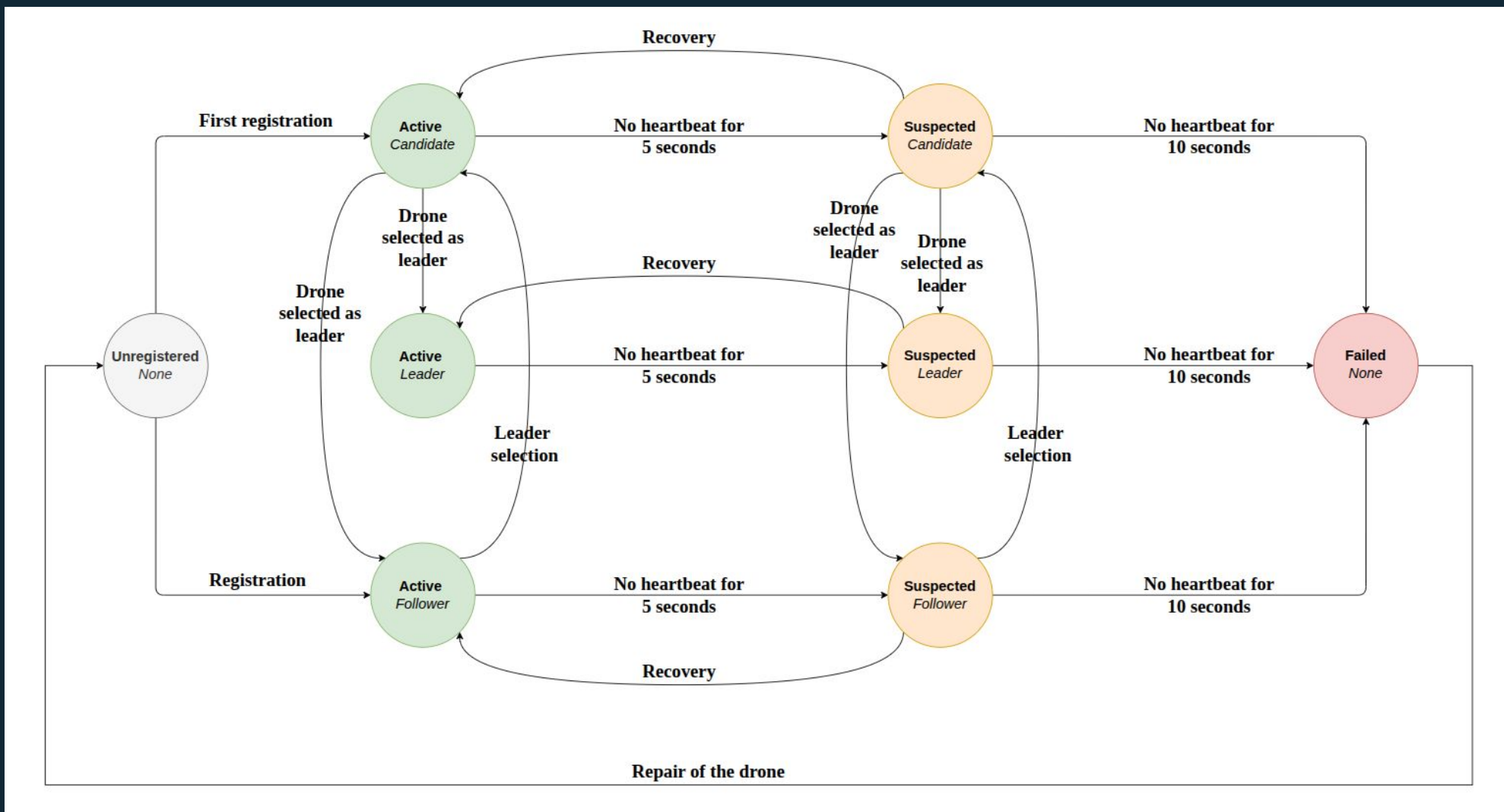
The overall state of each drone agent is determined by combining both state machines into a unified dual-state tuple (**S1**, **S2**), where:

- **S1 = Health State** (Unregistered, Active, Suspected, Failed)
- **S2 = Swarm Role** (None, Candidate, Follower, Leader)

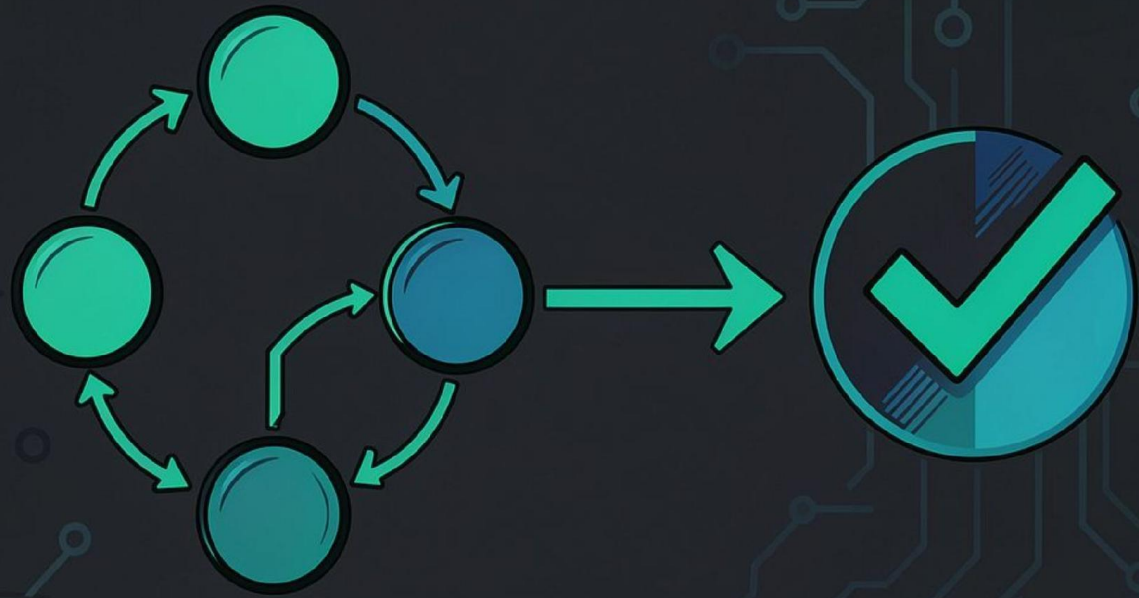
Not all combinations of (S1, S2) are valid. The complete state transition model defines which state pairs are reachable and what events trigger transitions between them. For example:

- **A Failed drone (S1=Failed) must have S2=None (no role)**
- **Only Active drones (S1=Active) can be Leaders, Followers, or Candidates**
- **Unregistered drones (S1=Unregistered) must have S2=None until they transition to Active**

The complete state transition diagram below illustrates all valid state pairs and the transition events that move the system between them:



This unified model ensures that the swarm maintains consistency: health monitoring and role management are coordinated, preventing invalid states and ensuring robust fault tolerance.



Formal model verification with Kani

Verifying *FSM* properties — Kani used to verify FSM
correctness

What is Kani?

Kani is an open-source bit-precise model checker developed primarily for the Rust programming language. It uses symbol-based formal verification techniques to exhaustively analyze code, going far beyond traditional unit tests or compiler checks.

Kani serves to mathematically prove that a piece of Rust code satisfies specific safety and correctness properties for any possible input (exhaustive verification), rather than just testing a few sample cases. It analyzes code by leveraging logical solvers (SAT/SMT engines via CBMC) to spot logical bugs or undefined behaviors.

Main applications:

Verification of unsafe blocks

- Although Rust natively guarantees memory safety, the use of unsafe blocks bypasses compiler checks
- Kani is ideal for analyzing these critical sections and ruling out memory bugs
- Essential for systems where memory safety is critical

Panic and overflow checking

- Automatically detects whether an operation can cause a runtime panic
- Examples: calling `unwrap()` on `None` values, arithmetic overflows
- Prevents unexpected runtime failures

Why Kani is So Useful



Total coverage (100%)

- Unlike traditional tests that only check inputs written by the programmer, Kani explores all possible execution paths
- Provides a concrete counterexample if it finds a violation
- Guarantees exhaustive verification across all scenarios



Higher confidence in critical systems

- Helps achieve extremely high reliability standards in complex industrial fields
- Applications: automotive, aerospace, embedded systems
- Where a runtime error can be catastrophic
- Provides mathematical proof of correctness



Native integration

- Integrates directly with Rust's package manager (cargo kani)
- Makes the execution of formal checks smooth within modern development workflows
- Seamless CI/CD integration
- No external tools or complex setup required

Events in the FSM verified with KANI:

Event Name (SwarmEvent)	Event Description	Previous State → Next State
RegistrationSuccess { has_active_leader }	Triggered when a drone completes registration. If no leader exists: Drone becomes a Candidate. If leader exists: Drone becomes a Follower.	if has_active_leader == false: <i>(Unregistered, None) → (Active, Candidate);</i> if has_active_leader == true: <i>(Unregistered, None) → (Active, Follower)</i>
LeaderSelection { is_chosen }	Applied to Candidate drones during leader election. If selected (is_chosen == true): Drone becomes Leader. If not selected (is_chosen == false): Drone becomes Follower.	if is_chosen == true: <i>(Active, Candidate) → (Active, Leader);</i> if is_chosen == false: <i>(Active, Candidate) → (Active, Follower)</i>
HeartbeatTimeout {elapsed_seconds}	Applied to drones when heartbeats are missing. If (5 < elapsed_seconds < 10) drone is not yet considered failed, but it is suspected. If (elapsed_seconds >= 10) drone is considered failed and is despawned from Gazebo,	If (5 < elapsed_seconds < 10): <i>(Active, S2) → (Suspected, S2);</i> If (elapsed_seconds >= 10) <i>(Active, S2) → (Failed, S2)</i>
HeartbeatRecovered	Heartbeat received within 10 seconds	<i>(Suspected, S2) → (Active, S2)</i>
LeaderFailedReelectionTrigger	The active Leader drone fails; triggers all Followers to enter election candidate status.	<i>(Active, Follower) → (Active, Candidate);</i> <i>(Suspected, Follower) → (Suspected, Candidate)</i>
RepairCompleted	Container unpaused/repaired; resets drone to initial state ready for re-registration.	<i>(Failed, None) → (Unregistered, None)</i>

Kani's Logical Workflow

Kani verifies safety properties through a structured three-phase process — from generating all possible states to mathematically proving (or disproving) a property holds.



Phase 1: Setup

`kani::any()` + `kani::assume(...)`

- `kani::any()` generates *all possible* values and states
- `kani::assume(...)` filters out invalid or absurd combinations
- Only **valid initial states** remain on the table



Phase 2: Execution

Code Execution / State Transition Loop

- Kani runs the code through **state transitions** (or loop iterations)
- Each valid state from Phase 1 is used as a starting point
- All reachable execution paths are explored



Phase 3: Verification

`kani::assert(...)`

- Kani checks if *any single trace* makes the property **FALSE**
-  **VERIFIED** — property holds in ALL valid cases
-  **ERROR** — stops and shows exact counterexample

 **Key insight:** Kani performs *exhaustive* mathematical verification — not random sampling. If no counterexample exists across all valid paths, the property is guaranteed to hold.

Kani Implementation:

The implementation of a formal model verification with Kani is based on verifying specific properties to ensure robustness and correctness of the FSM analyzed. We can make this by using two very important functions:

kani::assume — Defines valid pre-conditions of the system in formal logic. It tells Kani to consider only sensible initial configurations (e.g., if the starting state is `Unregistered` or `Failed`, assume initial role `S2` is `None`). This ensures states always start from valid starting conditions.

```
// Assume valid initial configuration: if S1 is Unregistered or Failed, S2 must be None
if current.s1 == Fsm1State::Unregistered || current.s1 == Fsm1State::Failed {
    kani::assume(current.s2 == Fsm2State::None);
}
```

This is an example of how to use **kani::assume**, here I'm using it to make an assumption in the block of code used to prove the property of **Inactive role invariant** (Harnesses verification function in the code)

Kani Implementation:

kani::assert — Defines the safety property to be proven. It tells Kani to mathematically verify a specific condition (e.g., after applying an event, if the new state S1 is `Unregistered` or `Failed`, verify that role S2 is **mandatorily** `None`). If any single event or combination of events violates this rule, Kani immediately stops and reports an error with the exact counterexample.

```
// Safety assertion: after ANY event, Unregistered or Failed MUST have S2 == None
if next.s1 == Fsm1State::Unregistered || next.s1 == Fsm1State::Failed {
    kani::assert(next.s2 == Fsm2State::None, "S2 must be None when S1 is Unregistered or Failed");
}
```

This is an example of how to use **kani::assert**, kani checks whether there is even a single trace/combination among the valid ones that makes the property FALSE.

Verified Properties:

Located in the `Kani_verify/` directory, proof harnesses written for the Kani model checker mathematically verify the following properties:

Property	Description	Significance
Inactive Role Invariant	An Unregistered or Failed drone must ALWAYS have S2 = None.	Ensures that drones without operational status cannot hold any swarm role, preventing invalid state combinations.
Suspected Role Preservation	Entering Suspected or recovering back to Active preserves the S2 role unchanged.	Guarantees that temporary health degradation does not affect role assignment, maintaining swarm structure during transient failures.
Single Leader & Recovery Invariant	In any valid swarm state, at most one active Leader exists. Upon Leader failure, re-election guarantees a single new Leader without violating the invariant when repaired drones rejoin.	Ensures swarm coordinator has one leader and shows that recovery re-election is never triggered prematurely before the 10-second no-heartbeat bound is exceeded.

An abstract illustration on the left side of the slide. It features two drone-like agents, each represented by a cube with four propellers. They are positioned on a floor made of a perspective grid. The background shows stylized, jagged lines representing hills or mountains. The entire scene is rendered in a dark blue color palette with light blue and white lines.

Drone Agent Implementation

Drones — Each drone agent is an independent component communicating via asynchronous messages

Drone Agent Architecture:

`drone_agent_wrapper.py` is the ROS 2 node (written in Python) that represents the onboard intelligence of an individual drone within the swarm. It is not the centralized coordinator, but rather the Individual Agent running inside each drone's container/process (corresponding to the Drone_1, Drone_2, ..., Drone_N blocks in the architecture diagram).

Key Responsibilities:



Heartbeat & Registration (Telemetry)

- Periodically publishes the drone's online status and current coordinates
- Allows the Rust Coordinator to detect and track the drone

Collision Avoidance

The drone agent implements a sophisticated local control loop running at 20 Hz that combines attractive forces toward the target destination with repulsive forces from neighboring drones to achieve safe, collision-free navigation.



Local Navigation

- Receives 3D target destinations (x, y, z) via `/swarm/goto` topic from Rust Coordinator
- Enables the drone to reach the point and perform steady hovering



3D Decentralized Obstacle Avoidance (Artificial Potential Fields - APF)

- Applies 3D repulsive forces to prevent collisions between neighboring drones



Layered Takeoff/Transit Strategy

- Manages differentiated transit altitudes (e.g., vertical ascent before horizontal movement)
- Prevents physical collisions during takeoff and formation phase

ROS 2 Integration with Drone Wrapper

Architecture & Inheritance from the Framework (rclpy)

In the `drone_wrapper` file it is used the **official ROS 2 client library for Python, `rclpy`**. The main class inherits directly from the base ROS 2 `Node` class:

```
class DroneAgentWrapper(Node):  
    def __init__(self):  
        drone_id = os.environ.get("DRONE_NAME", "drone_1")  
        super().__init__(f"{drone_id}_agent")
```

This transforms the Python script into a **native node of the ROS 2 graph**, capable of:

- Publishing and subscribing to topics
- Managing asynchronous timers via the executor (`rclpy.spin(node)`)

Quality of Service (QoS Profile) Configuration

To ensure **reliable communication** with the Rust Coordinator and the Fast-DDS network, a custom **QoS (Quality of Service)** profile is configured:

```
qos_profile = QoSProfile(  
    reliability=ReliabilityPolicy.RELIABLE,  
    history=HistoryPolicy.KEEP_LAST,  
    depth=10  
)
```

This ensures that **critical messages** — such as registration and formation commands — are **not lost** over the DDS network.

Drone's Communication Topics:

Understanding how drones and the coordinator communicate is key to the system's distributed nature.

1

/swarm/register

Drone → Coordinator

New drone registration and initialization

2

/swarm/heartbeat

Drone → Coordinator

Real-time telemetry and position updates

3

/swarm/goto

Coordinator → Drone

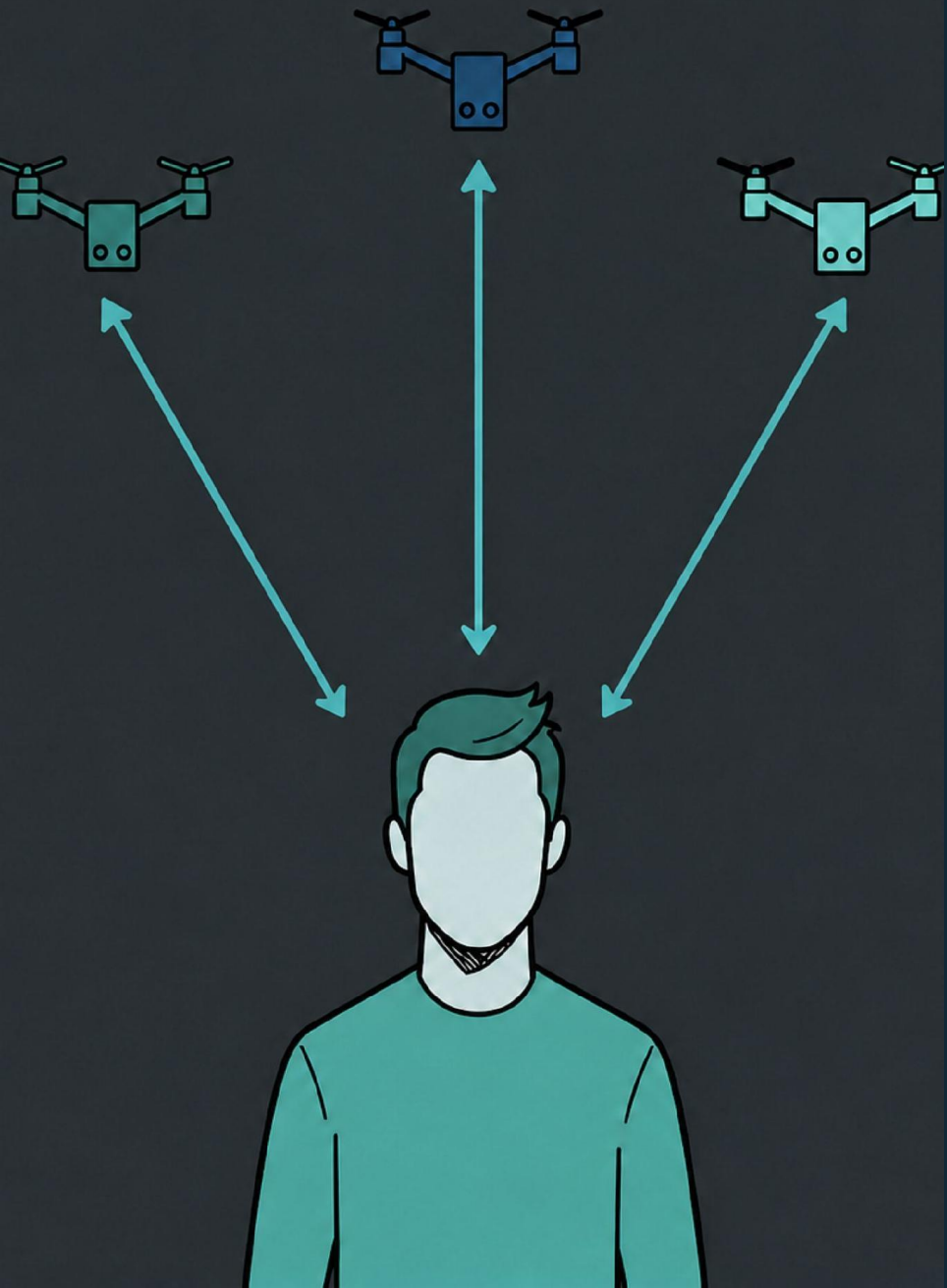
Target coordinates for autonomous navigation

4

/drone_X/odometry

Gazebo → Drone

Avoid collisions



Rust Coordinator

Coordinator in Rust — The swarm is managed by a separate coordinator

The Rust Coordinator: 4 Key Roles

We can see the coordinator in this project like the "central brain", managing several key aspects of the entire simulation.



Registry & Swarm Telemetry

Listens for drone registrations, monitors health status and real-time 3D positions. Maintains a centralized drone database.



Formally Verified FSM Execution

Manages global swarm state transitions (Unregistered → Active → Suspected → Failed). Safety rules are formally verified with Kani.



Leader Election & Re-election

Determines swarm Leader using dynamic proximity/density-based algorithms. Ensures automatic re-election upon leader failure.



Formation Computation & Assignment

Translates global commands into individual 3D coordinates and manages altitude layering for collision prevention. Dispatches `/swarm/goto` commands.

Coordinator Architecture: Centralized Brain of the Swarm

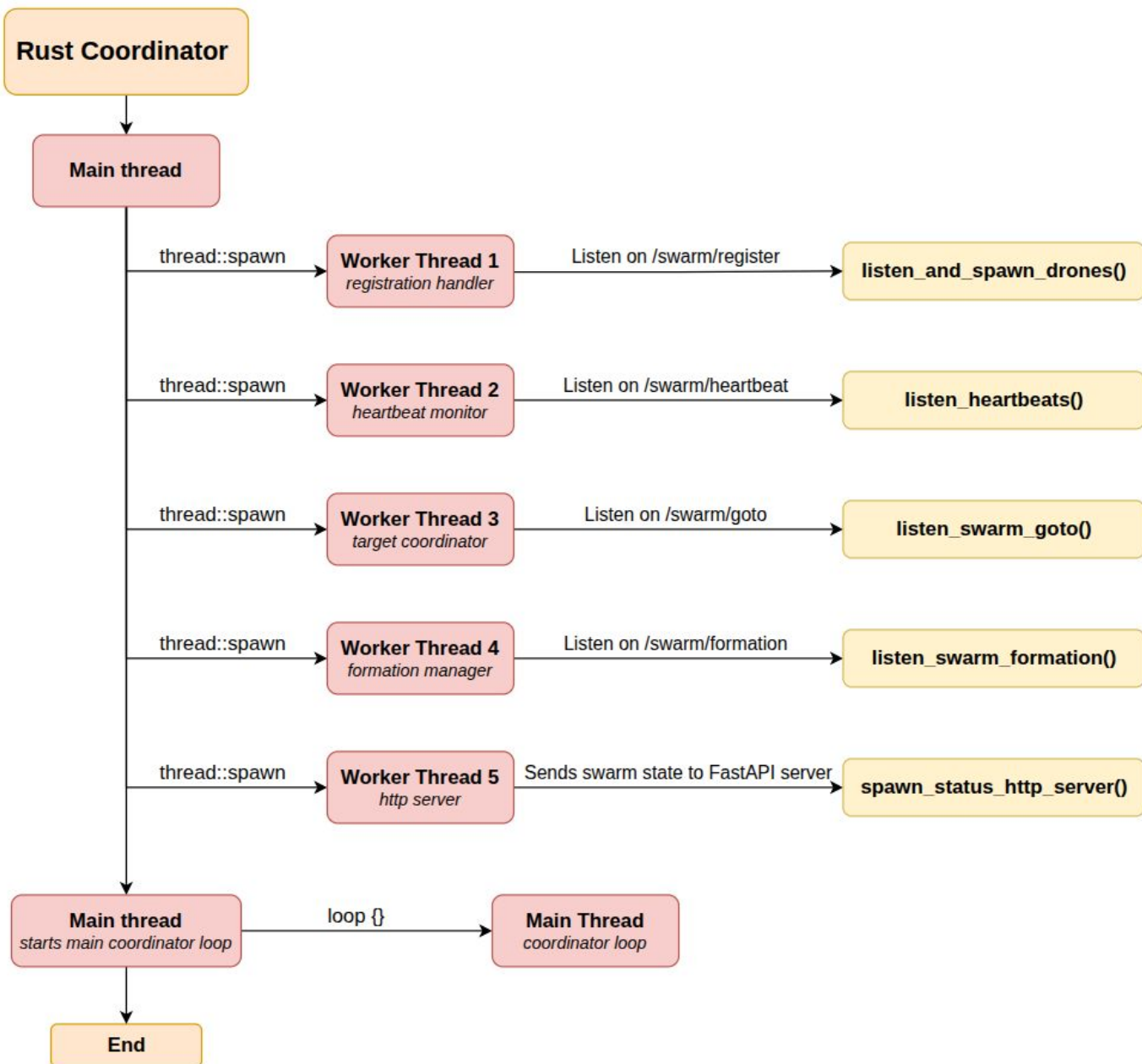
What the Coordinator Does:

- Centralized decision-making authority
- Receives all drone telemetry and heartbeats
- Executes formally verified FSM logic
- Computes and assigns flight formations
- Manages leader election and fault recovery

How It Communicates:

- Listens to: `/swarm/register`, `/swarm/heartbeat`
- Sends to: `/swarm/goto` (target coordinates)
- Maintains: Global swarm state (`Arc<Mutex<T>>`)
- Runs: Multi-threaded listener hierarchy
- Ensures: Non-blocking asynchronous execution

Thread Architecture:



Main Thread - Coordinator Loop



- Monitors heartbeat timeouts (5 seconds for Suspected, 10 seconds for Failed)
- Executes leader election algorithms when needed
- Triggers dynamic swarm re-formations based on current state
- Synchronizes with worker threads via thread-safe shared state

Worker Thread 1: Registration Handler



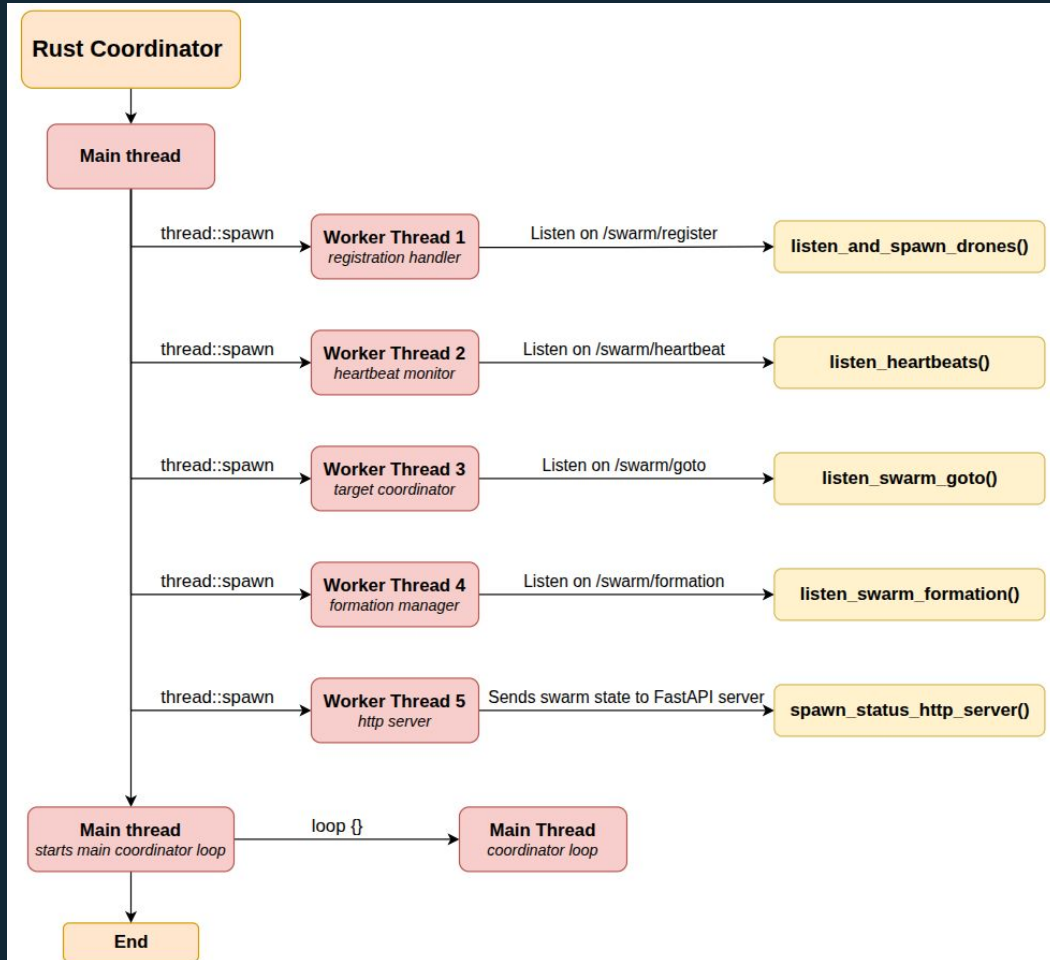
- Listens continuously to `/swarm/register` topic
- Handles new drone registrations and initialization
- Triggers Gazebo spawning of new drone instances
- Updates `SwarmCoordinatorState` with new drone metadata

Worker Thread 2: Heartbeat Monitor



- Listens continuously to `/swarm/heartbeat` topic
- Receives real-time 3D telemetry and position updates
- Updates drone timestamps for timeout detection
- Maintains current position data for collision avoidance

Thread Architecture:



Worker Thread 3: Target Coordinator



- Listens continuously to `/swarm/goto` topic
- Tracks individual coordinate target vectors for each drone
- Coordinates with formation algorithms
- Ensures each drone receives its assigned destination without blocking other operations

Worker Thread 4: Formation Manager



- Listens continuously to `/swarm/formation` topic
- Triggers global swarm pattern algorithms
- Manages formation transitions and reconfigurations
- Broadcasts formation commands to all drones
- Handles dynamic formation changes in response to swarm state

Worker Thread 5: Http server



- Exposes an HTTP REST endpoint on coordinator port 8080.
- Reads FSM state thread-safely from the shared Mutex.
- Returns structured JSON status responses to the FastAPI backend.

Integration with Main Thread:

- All worker threads update the shared `Arc<Mutex<SwarmCoordinatorState>>`
- Main thread reads the updated state every 2 seconds
- Formation and navigation updates are immediately available to the coordinator
- No blocking I/O operations interfere with FSM state machine execution



How to spawn a Drone

Spawning drones — Full logical steps to make drones appear in the simulation

Step 1: Drone Container Startup & Environment Configuration

This command triggers the entire project, spawning N drones: `docker compose up --scale drone=N`

O1

Docker Engine instantiates N separate containers

(`drone_1`, `drone_2`, `drone_3`, etc.)

O2

Entrypoint script configures ROS 2 / Fast-DDS network environment

O3

Dynamically calculates initial 3D ground spawn coordinates (X, Y, Z)

O4

Each drone container is isolated and independent

Files & Technology:

- `entrypoint_drone_agent.sh`
- Docker Compose, Bash Scripting, Fast-DDS Discovery Server

Step 2: ROS 2 Node Initialization & Registration Request

Each drone **announces itself** to the coordinator by publishing a registration message. A 2-second timer triggers the payload, sending the drone's ID and initial coordinates to the shared `/swarm/register` topic via Fast-DDS with RELIABLE QoS.

```
def publish_registration(self):
    payload = {
        "drone_id": self.drone_id, # example: "drone_1"
        "x": self.spawn_x, "y": self.spawn_y, "z": self.spawn_z
    }
    msg = String()
    msg.data = json.dumps(payload)
    self.registration_pub.publish(msg) # Publish on /swarm/register
```

Steps:

- Container boots
- Construction of JSON payload
- ROS 2 Node publishes on `/swarm/register`

Files & Technology:

- `drone_agent_wrapper.py` & `entrypoint_drone_agent.sh`
- rclpy, Fast-DDS

Step 3: Coordinator intercepts registration request

```
# Listener in background on /swarm/register
let mut child = Command::new("ros2")
    .args(&["topic", "echo", "/swarm/register", "std_msgs/msg/String"])
    .stdout(Stdio::piped()).spawn().unwrap();

# Parsing of JSON ed extraction of drone ID and spawn coordinates
if let Ok(payload) = serde_json::from_str::(<RegistrationPayload>(&unescaped_json) {
    # Inserts the drone in the shared space and executes the spawn on Gazebo
    let spawn_success = spawn_drone_in_gazebo(&payload);
}
```

Steps:

- Coordinator actively listens for drone registrations
- Multi-threaded architecture enables concurrent listening
- JSON parsing extracts critical drone metadata
- call function `spawn_drone_in_gazebo()`

Files & Technology:

- `main.rs`
- Rust (`std::process::Command`, `std::thread`), Serde JSON

Step 4: Dynamic 3D SDF Model Generation (Namespace Injection)

The Rust Coordinator dynamically generates 3D SDF (Simulation Description Format) models for each drone, ensuring isolated environments within the simulation.

```
let robot_namespace = format!("/model/{}", payload.drone_id);

// Read the template of the model and injects its unique namespace depending on the ID
let sdf_template = fs::read_to_string("/app/models/x3_uav/model.sdf").unwrap();
let sdf_instance = sdf_template.replace("__ROBOT_NAMESPACE__", &robot_namespace);

// Saves the SDF file
let instance_path = format!("/app/spawned/{}/_model.sdf", payload.drone_id);
fs::write(&instance_path, sdf_instance).unwrap();
```

Steps:

- Rust Coordinator reads generic 3D SDF model template (x3_uav/model.sdf)
- Clones the template for each drone
- Replaces __ROBOT_NAMESPACE__ placeholder with unique namespace
- Example: /model/drone_1, /model/drone_2, etc.
- Each drone gets isolated topic arguments and plugins inside Gazebo

Files & Technology:

- `main.rs`
- Rust File I/O (`std::fs`), SDFFormat (SDF 3D Specification)

Step 5: Invoking ros_gz_sim create (Gazebo Simulation Spawn)

```
let output = Command::new("ros2")
    .args(&[
        "run", "ros_gz_sim", "create",
        "-world", &world,
        "-name", &payload.drone_id,
        "-x", &payload.x.to_string(),
        "-y", &payload.y.to_string(),
        "-z", &payload.z.to_string(),
        "-file", &model_path,
    ])
    .output();
```

Steps:

- Rust Coordinator executes ros_gz_sim create CLI utility
- Passes Gazebo world name, target 3D coordinates (X, Y, Z)
- Passes drone name and generated SDF file path
- Network command instructs Gazebo simulator engine
- Gazebo instantiates 3D drone mesh inside active world

Files & Technology:

- `main.rs`
- `ros_gz_sim` (ROS 2 - Gazebo Harmonic Bridge), Gazebo Sim Service Client

Step 6.1: Formally Verified FSM Update & Telemetry Stream Activation

```
if spawn_success {  
    let mut guard = swarm_state.lock().unwrap();  
    if let Some(drone) = guard.drones.get_mut(&drone_id) {  
  
        // Executes transition (Event 1: RegistrationSuccess)  
        drone.state = transition(  
            drone.state,  
            SwarmEvent::RegistrationSuccess { has_active_leader },  
        );  
    }  
}
```

Steps:

- If spawn call has success the Coordinator locks the shared state and triggers the Kani-verified state transition function moving the drone state from `Unregistered` to `Active`.
- At this point, the drone physically appears in Gazebo, the `ros_gz_bridge` begins publishing ground-truth odometry, and the Python agent takes control of flight navigation.

Step 6.2: Candidate drones

```
// Event 1 FSM Transition logic (lib.rs)
SwarmEvent::RegistrationSuccess { has_active_leader } => {
    if current.s1 == Fsm1State::Unregistered {
        let s2 = if has_active_leader { Fsm2State::Follower } else { Fsm2State::Candidate };
        DroneState { s1: Fsm1State::Active, s2 }
    } else { current }
}
```

Steps:

- Once all drones are spawned, at the very beginning there are no active leaders.
- The Rust Coordinator applies **FSM Event 1** (`RegistrationSuccess`). Since no active leader exists yet (`has_active_leader == false`), each registering drone transitions from (`Unregistered`, `None`) to (`Active`, `Candidate`), making them eligible participants for the upcoming election.

Files & Technology:

- `main.rs` & `lib.rs` (Kani_verify)
- Kani Formal Verification Engine, Rust State Machine (transition)

Step 7: Leader election & FSM role assignment

```
// Event 2 FSM Transition logic (lib.rs)
for (id, drone) in state.drones.iter_mut() {
    if drone.state.s2 == Fsm2State::Candidate {
        let is_chosen = id == &chosen_leader_id;
        drone.state = transition(drone.state, SwarmEvent::LeaderSelection { is_chosen });
    }
}
```

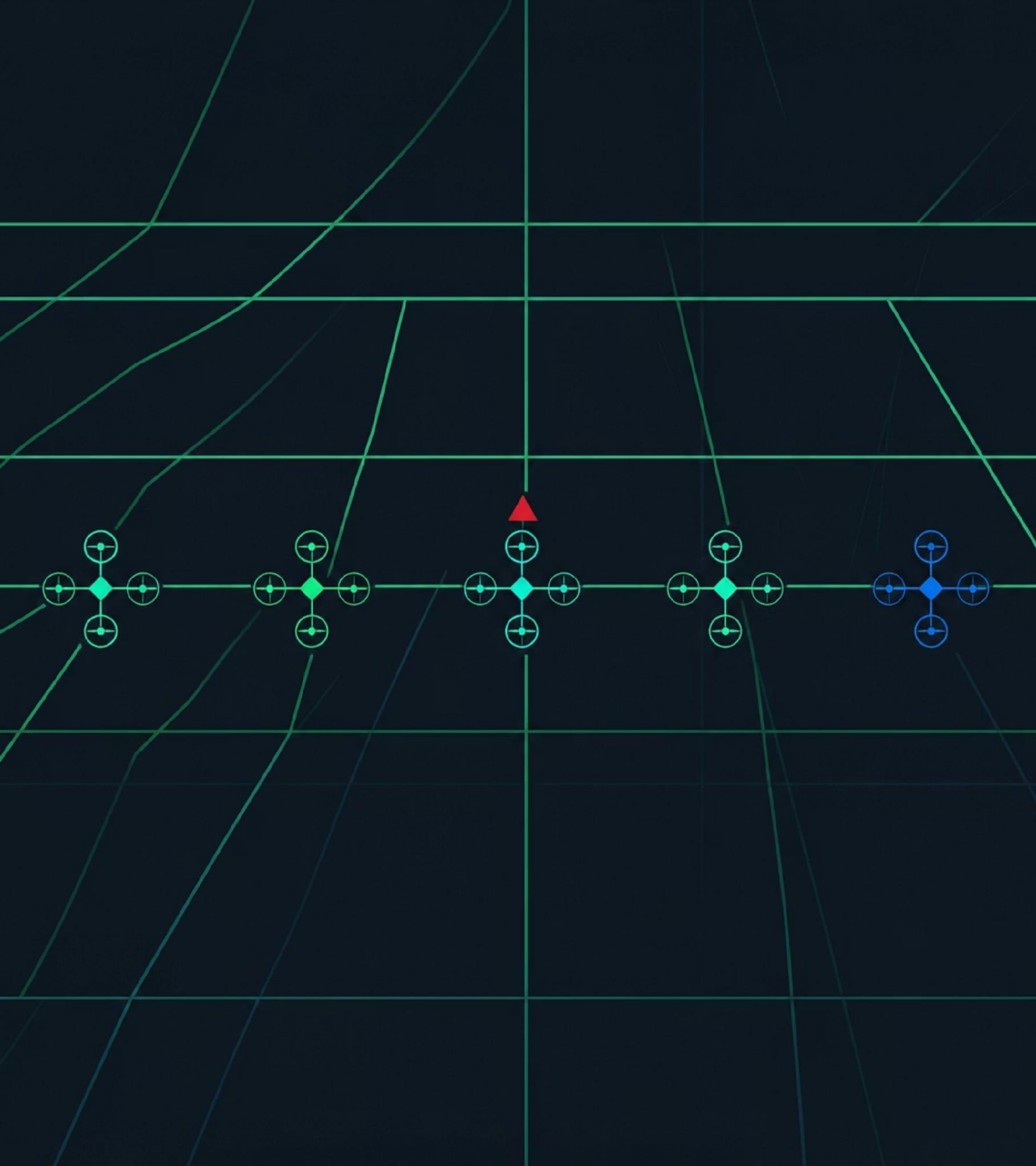
Steps:

- Once the election algorithm picks a winner (e.g., `drone_1`), the Coordinator iterates through all Candidate drones and applies **FSM Event 2** (`LeaderSelection`):
- For the chosen candidate (`is_chosen == true`): State transitions to (`Active, Leader`).
- For all non-chosen candidates (`is_chosen == false`): State transitions to (`Active, Follower`).

The Leader election is officially complete, now the drones are spawner and are on the ground, waiting for the order to arrange themselves in formation to take off

Files & Technology:

- `main.rs` & `lib.rs` (Kani_verify)
- Kani Formal Verification Engine, Rust State Machine (transition)



How to put drones in formation

Line Formation — Full logical steps to make drones
arrange in formation

Step 1: Line formation request

```
# FastAPI endpoint triggering container exec (main.py)
@app.post("/api/swarm/formation/line")
def trigger_line_formation():
    client = get_docker_client()
    coordinator_container = get_coordinator_container(client)

    cmd = [
        "bash", "-c",
        'export ROS_DISCOVERY_SERVER=172.28.0.10:11811 && source /opt/ros/jazzy/setup.bash'
        && '
        'ros2 topic pub --once /swarm/formation std_msgs/msg/String "data:
        \'{{\\"formation\\": \\"line\\"}}\''
        ]
    exec_result = coordinator_container.exec_run(cmd) # Execute ROS 2 pub command inside
    container
    return {"status": "success", "output": exec_result.output.decode("utf-8")}
```

Steps:

- The frontend sends an HTTP POST request to the `/api/swarm/formation/line` endpoint. The FastAPI server interacts with the Docker SDK (`docker.from_env()`) to invoke a `ros2 topic pub --once` CLI command inside the running Coordinator container via Docker Engine `exec_run`.
- Payload published on `/swarm/formation` topic over the Fast-DDS Discovery Server network (IP `172.28.0.10:11811`).
- The CLI command publishes the one-shot JSON payload onto the ROS 2 network. The Fast-DDS Discovery Server routes this publication towards all subscribers listening to topic `/swarm/formation`.

Files & Technology:

- `main.py` & `app.js`
- FastAPI, Docker SDK for Python (`exec_run`), JavaScript (Fetch API).

Step 2: Formation Request Interception & Geometric Slot Calculation

```
# Shell command executed inside the Coordinator container
ros2 topic pub --once /swarm/formation std_msgs/msg/String "data: '{\"formation\":  
\"line\"}'"
```

```
// Anchor formation center to the Leader's coordinates (center_x, center_y)
let (center_x, center_y) = (leader_drone.x, leader_drone.y);
let safety_distance = 2.0;

// Compute symmetric follower target Y offsets (center_y +/- k * safety_distance)
for idx in 0..num_followers {
    let pair_index = ((idx / 2) + 1) as f64;
    let sign = if idx % 2 == 0 { -1.0 } else { 1.0 };
    target_y_list.push(center_y + sign * pair_index * safety_distance);
}
```

Steps:

- The coordinator receives the request to arrange drones in formation (e.g., "line" on topic /swarm/formation)
- the Rust Coordinator locks the active Leader at the center and symmetrically computes target 3D offsets for all active Followers.

Files & Technology:

- main.rs
- ROS 2 Topic Subscriber

Step 3: Asynchronous Target Dispatch

```
// Coordinator dispatches target payload JSON over ROS 2 topic /swarm/goto
let json_str = format!(r#"{{"drone_id": "{}", "x": {}, "y": {}, "z": {}}}"#, drone_id, tx,
ty, tz);
Command::new("ros2").args(&["topic", "pub", "-t", "3", "/swarm/goto", "std_msgs/msg/String",
&data_arg]).output();
```

Steps:

- The Coordinator dispatches individual target commands via `ros2 topic pub` on topic `/swarm/goto`
- Each drone's Python agent receives its assigned target
- Each drone drives its motors using a P-Controller at 20 Hz, while applying Artificial Potential Fields (APF) to maintain safe distances from peer drones avoiding collisions

Files & Technology:

- `main.rs` & `drone_agent_wrapper`
- ROS 2 Topic Publisher, Python P-Controller, 3D Artificial Potential Fields (APF).

Step 4: Collision avoidance

Steps:

- To prevent mid-air collisions during formation changes, the Coordinator assigns staggered transit flight altitudes 4.8m for even-indexed followers, 5.6m for odd-indexed followers). Adjacent drones never cross horizontal space at the same height.
- **APF (Artificial Potential Fields):** each drone applies a 3D repulsive forces to prevent collisions between neighboring drones

Files & Technology:

- `main.rs`
- Altitude Layering Collision Avoidance Protocol, Rust State Matching.



Failure and repair

Fail and Recovery — Full logical steps to simulate failure of a drone and recovery

Scenario 1: **Follower** Drone Failure

1

Failure Detection

- Heartbeat timeout (10s)
- Drone marked as Failed in FSM
- Despawnd from Gazebo

2

State Persistence

- Coordinator maintains target coordinates in memory
- Previous 3D position stored for every drone

Scenario 2: **Leader** Drone Failure & Reconfiguration



1) FSM Role Reset

Leader drone fails (detected via heartbeat timeout). All active Followers transition to Candidate state. Drones become eligible for new leader election.



2) Smart Leader Election

Algorithm considers spatial proximity to failed leader's slot. Selects candidate from denser wing (side with more drones). Ensures balanced formation.



3) Ripple-Shift Re-formation

New Leader moves to center position. Opposite wing remains stationary (no unnecessary moves). Vacant wing performs cascade shift inward by 2.0m safety distance. Formation automatically rebalances.

Step 1: User simulates the failure

```
import docker

# 1. Connects to the host Docker Daemon via /var/run/docker.sock
client = docker.from_env()

@app.post("/api/drone/{drone_id}/pause")
def pause_drone(drone_id: str):
    # 2. Searches for the container matching target drone_id (e.g., "drone_1")
    for container in client.containers.list(all=True):
        if drone_id in container.name:
            target_container = container
            break

    # 3. Invokes Python Docker SDK pause method
    target_container.pause() # <--- Equivalent to executing 'docker pause <container_id>'
    return {"status": "success"}
```

Steps:

- When the users click this button, the JavaScript code (app.js) intercepts the click event and sends an HTTP POST request to the REST endpoint: POST /api/drone/drone_1/pause
- The FastAPI server utilizes the official docker library (Docker SDK for Python) to communicate directly with the Docker daemon running on the host OS.

Files & Technology:

- main.py & app.js
- FastAPI, Docker SDK for Python (exec_run), JavaScript (Fetch API).

Step 2: Docker pauses the container

The call `target_container.pause()` is functionally identical to running the CLI command:

```
docker pause implementation-drone-N
```

At the Linux Operating System level:

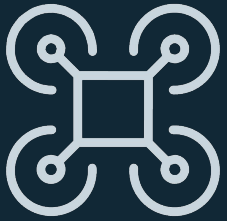
- It issues a `SIGSTOP` signal to all running processes inside that specific container.
- The Python process (`drone_agent_wrapper.py`) inside that container is **instantly frozen in its current execution state**.
- **IMMEDIATE EFFECT:** The drone stops broadcasting messages across the ROS 2 / Fast-DDS bus. Odometry streaming and heartbeats on `/swarm/heartbeat` cease entirely.

Step 3: How rust coordinator reacts to docker pause

Because the container process is frozen:

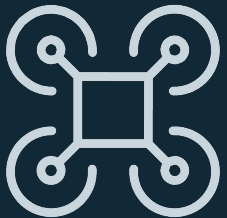
1. After 5 seconds of missing heartbeats, the Rust Coordinator's main event loop detects `elapsed_seconds > 5s` and triggers FSM transition from **Active** to **Suspected**.
2. After 10 seconds, the critical failure threshold is reached (`elapsed_seconds > 10s`) and triggers FSM transition from **Suspected** to **Failed**.
3.
 - Applies FSM event `DroneFailed` (**Failed** state).
 - Despawns the drone mesh from 3D Gazebo (`despawn_drone_from_gazebo`).
 - If the failed drone was the **Leader**, it triggers dynamic Leader Re-election and wing ripple-shift formation re-alignment!

Drone Repair & Re-integration: Two Scenarios



Scenario 1: No Reconfiguration

Formation remains unchanged while drone is offline. Upon repair, the drone automatically returns to its original slot. The coordinator sends it back to its previous position.



Scenario 2: Formation Reconfigured

Leader fails and formation is reconfigured with ripple-shift. Original slot no longer exists. Upon repair, the drone is assigned to one of the formation ends and integrates at the edge.

Repair:

```
@app.post("/api/drone/{drone_id}/unpause")
def unpause_drone(drone_id: str):
    """Unpause target drone container to simulate repair."""
    client = get_docker_client()
    target_container = None

    # Search for container by drone ID
    for container in client.containers.list(all=True):
        if drone_id in container.name or container.name.endswith(drone_id.replace("_", "-")):
            target_container = container
            break

    if not target_container:
        raise HTTPException(
            status_code=404, detail=f"Drone container for '{drone_id}' not found."
        )

    try:
        # Unfreeze container process using Python Docker SDK
        target_container.unpause() # <--- Equivalent to 'docker unpause <container_id>'
        return {"status": "success", "message": f"Unpaused {target_container.name}"}
    except Exception as err:
        raise HTTPException(status_code=500, detail=str(err))
```

Steps:

- When you click "**Simulate Repair**", it sends POST `/api/drone/drone_1/unpause`.
- FastAPI executes `target_container.unpause()` (equivalent to `docker unpause`)
- The container process is unfrozen (`SIGCONT`) and the process resumes execution, re-broadcasts on `/swarm/register`.
- The Rust Coordinator applies FSM event `RepairCompleted` (`Unregistered` → `Active`), respawns the 3D model in Gazebo, and the drone autonomously navigates back to its designated formation slot!

Summary: Key Results



Distributed & Fault-Tolerant Swarm Architecture

- Coordinated swarm based on ROS 2 & Docker
- Actor Model with asynchronous messaging



Formally Verified FSM Correctness

- 100% formal verification using Kani Model Checking
- Safety guarantees for state transitions



Decentralized 3D Navigation & Collision Avoidance

- Real-time APF (Artificial Potential Fields)
- Autonomous obstacle avoidance
- Smart leader selection and reconfiguration

Thanks for Your Attention

Student: Francesco Raimondo

Course: Engineering in Computer Science, University of Messina

GitHub Repository: <https://github.com/francesco-raimondo/iiot-drone-coordinator-rust-docker>